

DOCUMENTO DE REQUISITOS

Motor de Busca Simplificado utilizando Estruturas de Dados

Disciplina: Estruturas de Dados I
Projeto Semestral 2026

0. Visão Geral e Arquitetura do Sistema

Este documento descreve os requisitos de implementação do Motor de Busca Simplificado, organizado como um roteiro de desenvolvimento em ordem de dependência. Cada etapa deve ser concluída antes de iniciar a próxima.

O sistema é dividido em três etapas principais que se conectam em pipeline:

Etapa	Nome	Responsabilidade
A	ETL (Extract, Transform, Load)	Leitura, limpeza e tokenização dos documentos
B	Representação dos Dados	Indexação e armazenamento eficiente dos termos
C	Motor de Busca e Similaridade	Recepção de consulta, cálculo de score e exibição de resultados

Fluxo de dados:

Documentos .txt → [Fila ETL] → [Pilha de Transformações] → Texto tokenizado
Texto tokenizado → [Tabela Hash + Lista Estática] → Vocabulário global indexado
Vocabulário + Docs → [Listas Encadeadas Esparsas] → Representação vetorial

Consulta do usuário → ETL → Vetor consulta → [Produto Escalar]
→ [ABB] → Resultado

1. Estruturas de Dados a Implementar

Implemente todas as estruturas abaixo antes de começar as etapas funcionais. Elas são os blocos construtivos do sistema.

1.1 Fila (Queue) — FIFO

Objetivo: controlar a ordem de processamento dos documentos na etapa ETL.

Operação	Descrição
enqueue(doc)	Insere um documento no final da fila
dequeue()	Remove e retorna o documento do início da fila
isEmpty()	Retorna verdadeiro se a fila estiver vazia
peek()	Consulta o primeiro elemento sem remover

Regras de implementação:

- Implementar com lista encadeada dinâmica (não usar array estático)
- A ordem de processamento deve ser estritamente FIFO
- Uso: recebe doc1.txt, doc2.txt, doc3.txt → processa doc1 primeiro

1.2 Pilha (Stack) — LIFO

Objetivo: armazenar o histórico das transformações aplicadas a cada documento durante o ETL.

Operação	Descrição
push(estado)	Empilha o estado atual do texto
pop()	Desempilha e retorna o estado mais recente
isEmpty()	Retorna verdadeiro se a pilha estiver vazia
peek()	Consulta o topo sem remover

Regras de implementação:

- Cada transformação do texto deve gerar um push na pilha
- Sequência esperada de pushes por documento:
 - Push: texto bruto original (ex: "Dados, Estrutura!")
 - Push: texto em minúsculas sem pontuação (ex: "dados estrutura")

- Push: lista de tokens (ex: ["dados", "estrutura"])

1.3 Lista Encadeada Dinâmica com Nó Sentinela

Objetivo: representar cada documento como um vetor esparsos (somente termos presentes). Também usada internamente na Tabela Hash para tratamento de colisões.

Estrutura do nó:

```
struct No {
    int id_termo;    // ID global da palavra
    int frequencia; // quantas vezes o termo aparece no documento
    struct No* proximo;
};
```

Operação	Descrição
inserir(id, freq)	Insere ou atualiza a frequência de um termo na lista
buscar(id)	Retorna o nó com o id_termo informado, ou NULL
percorrer()	Percorre todos os nós (exceto sentinela)

Regras de implementação:

- O nó sentinela é o PRIMEIRO nó da lista (cabeça) — sem dado válido
- Os nós devem ser mantidos em ordem crescente de id_termo (facilita o cálculo do produto escalar)
- Cada documento tem sua própria lista encadeada
- Exemplo: documento "dados estrutura dados" → [1|2] → [2|1] → [SENTINELA]

1.4 Tabela Hash com Encadeamento

Objetivo: mapear cada palavra (string) para um ID numérico único global, com acesso médio $O(1)$.

Estrutura do nó da lista de colisão:

```
struct NoBucketHash {
    char* palavra;
    int id_global;
    struct NoBucketHash* proximo;
};
```

Operação	Descrição
inserir(palavra, id)	Insere a palavra e seu ID na posição hash(palavra)
buscar(palavra)	Retorna o ID da palavra, ou -1 se não encontrada

Operação	Descrição
funcaoHash(palavra)	Calcula o índice na tabela para a string dada

Regras de implementação:

- O valor retornado pela função hash NÃO é o ID da palavra — é apenas o índice do bucket
- Duas palavras podem ter o mesmo hash (colisão) → tratar por encadeamento com lista ligada
- O ID da palavra é um contador global incrementado a cada nova palavra inserida (1, 2, 3, ...)
- Exemplo: hash("dados")=5, hash("estrutura")=5 → posição 5 armazena ambas em lista

1.5 Lista Estática (Dicionário Global ID → Palavra)

Objetivo: fazer o caminho inverso da hash — dado um ID, retornar a palavra correspondente.

```
char* vocabulario[MAX_TERMOS]; // vocabulario[id] = "palavra"
```

Operação	Descrição
inserirPalavra(id, palavra)	Armazena a palavra na posição id do array
obterPalavra(id)	Retorna a palavra na posição id

Regras de implementação:

- Tamanho fixo definido por MAX_TERMOS (escolha um valor suficientemente grande, ex: 10000)
- O índice do array é exatamente o ID global da palavra
- Integração: Hash responde "qual é o ID de X?" | Lista Estática responde "qual é a palavra do ID Y?"

1.6 Árvore Binária de Busca (ABB) por Score

Objetivo: armazenar os documentos ordenados por relevância (score) após o cálculo de similaridade.

Estrutura do nó:

```
struct NoABB {
    int id_documento;
    float score;
    struct NoABB* esquerda;
    struct NoABB* direita;
};
```

Operação	Descrição
inserir(id_doc, score)	Insere um documento com seu score na ABB (chave = score)
percursoEmOrdem()	Percorre a árvore in-ordem (esq → raiz → dir), do menor para o maior score
percursoDecrescente()	Percorre a árvore in-ordem reverso (dir → raiz → esq), do maior para o menor

Regras de implementação:

- A chave de ordenação é o score de similaridade
- Scores maiores ficam à direita, scores menores à esquerda
- Para exibir do mais relevante ao menos relevante: percurso in-ordem REVERSO (direita → raiz → esquerda)
- Documentos com score = 0 (nenhum termo em comum) podem ser excluídos dos resultados

2. Etapa A — ETL (Extract, Transform, Load)

Responsável por ler os arquivos de texto brutos e transformá-los em listas de tokens normalizados, prontos para indexação. Esta etapa usa a Fila e a Pilha implementadas na Seção 1.

2.1 Requisito: Leitura e Enfileiramento dos Documentos

Passo a passo:

1. Receber os caminhos dos arquivos .txt como argumentos (ou ler de um diretório)
2. Para cada arquivo: criar uma estrutura de documento e chamar enqueue() na Fila
3. O processamento deve ocorrer na ordem de chegada (FIFO)

Saída esperada:

Fila: [doc1.txt] → [doc2.txt] → [doc3.txt]

2.2 Requisito: Pipeline de Transformações (com Pilha)

Para cada documento retirado da fila com dequeue(), aplicar as seguintes transformações na ordem abaixo, empilhando cada estado na Pilha:

Passo	Transformação	Exemplo
1	Texto bruto lido do arquivo	"Dados, Estrutura!"

Passo	Transformação	Exemplo
2 (push)	Conversão para minúsculas	"dados, estrutura!"
3 (push)	Remoção de pontuação	"dados estrutura"
4 (push)	Tokenização (split por espaços)	["dados", "estrutura"]

Regras:

- Cada push representa um estado intermediário do texto
- O topo da pilha ao final contém a lista de tokens — é este que será usado na Etapa B
- Pontuação a remover: . , ! ? : ; () [] { } " ' - _ \ / @ # \$ % & * = + < >

2.3 Requisito: Remoção de Duplicados Adjacentes (Recursividade)

Após a tokenização, aplicar uma função RECURSIVA que remove tokens duplicados adjacentes na lista.

Especificação da função:

```
tokens* removerDuplicadosAdjacentes(tokens* lista) {
    // Caso base: lista vazia ou com 1 elemento → retorna lista
    // Caso recursivo: se lista[0] == lista[1], remove lista[0] e chama
    recursão
    //
    //          senão, avança para lista[1] e chama recursão
}
```

- Entrada: ["dados", "dados", "estrutura", "estrutura"]
- Saída: ["dados", "estrutura"]
- A recursão é OBRIGATÓRIA — não usar laço iterativo para esta função

3. Etapa B — Representação dos Dados

Transforma os tokens produzidos pelo ETL em estruturas numéricas eficientes para busca. Usa a Tabela Hash, a Lista Estática e as Listas Encadeadas Esparsas.

3.1 Requisito: Construção do Vocabulário Global

Para cada documento processado na Etapa A, percorrer seus tokens e construir o vocabulário global:

4. Para cada token na lista de tokens do documento:
 - Chamar `tabela_hash.buscar(token)`

- Se retornar -1 (não existe): incrementar contador_global_id, chamar tabela_hash.inserir(token, contador_global_id) e vocabulario.inserirPalavra(contador_global_id, token)
- Se já existir: usar o ID retornado pela hash

Estado esperado após processar todos os documentos:

```
Hash: hash("dados")=5 → posição 5: ["dados", ID=1] → ["estrutura", ID=2]
      hash("fila")=12 → posição 12: ["fila", ID=3]
Lista Estática: [1]="dados" [2]="estrutura" [3]="fila" ...
```

3.2 Requisito: Representação Esparsa dos Documentos

Para cada documento, após a construção do vocabulário, criar sua lista encadeada esparsa de termos:

5. Inicializar a lista encadeada com o nó sentinela
6. Para cada token do documento:
 - Obter o ID global via tabela_hash.buscar(token)
 - Verificar se já existe um nó com esse ID na lista encadeada
 - Se sim: incrementar a frequência do nó existente
 - Se não: inserir novo nó [id_termo | frequencia=1]
7. Manter os nós ordenados por id_termo crescente

Exemplo:

```
Documento: "dados estrutura dados"
IDs: dados=1, estrutura=2
Lista: [1 | freq=2] → [2 | freq=1] → [SENTINELA]
```

Estrutura de controle dos documentos:

```
struct Documento {
    int id_documento;
    char* nome_arquivo;
    No* lista_termos; // lista encadeada esparsa
};

Documento todos_documentos[MAX_DOCS];
int total_documentos = 0;
```

4. Etapa C — Motor de Busca e Similaridade

Recebe a consulta do usuário, a converte em vetor esparsa, calcula a similaridade com cada documento e retorna os resultados ordenados por relevância.

4.1 Requisito: Processamento da Consulta

A consulta do usuário deve passar pelas mesmas transformações do ETL (Seção 2):

Passo	Ação	Resultado
1	Entrada do usuário	"dados estrutura"
2	ETL (minúsculas, sem pontuação, tokenização)	["dados", "estrutura"]
3	Converter tokens em IDs via tabela_hash.buscar()	dados→ID=1, estrutura→ID=2
4	Contar frequência de cada token	dados=1, estrutura=1
5	Criar lista encadeada temporária	[1 1] → [2 1] → [SENTINELA]

Regras:

- A lista encadeada da consulta é temporária — deve ser alocada no início da busca e liberada ao final
- Se um token da consulta não existir na tabela hash (palavra desconhecida), ignorá-lo
- Manter a lista da consulta ordenada por id_termo crescente (igual às listas dos documentos)

4.2 Requisito: Cálculo do Score (Produto Escalar)

Para cada documento, calcular o score de similaridade com a consulta usando o Produto Escalar entre os vetores esparsos:

Fórmula:

$$\text{Score} = \sum (\text{freq_consulta}[i] \times \text{freq_documento}[i]) \quad \text{para cada ID em comum}$$

Algoritmo de percurso simultâneo nas duas listas:

Situação	Condição	Ação
IDs iguais	id_consulta == id_documento	score += freq_consulta × freq_documento; avança AMBOS os ponteiros
ID consulta menor	id_consulta < id_documento	Termo não está no documento; avança só o ponteiro da consulta
ID documento menor	id_documento < id_consulta	Termo não está na consulta; avança só o ponteiro do documento

Condição de parada: quando qualquer um dos ponteiros atingir o nó sentinela.

Exemplo completo:

Consulta: [1|1] → [2|1] → [SENTINELA]

Documento: [1|2] → [2|1] → [SENTINELA]

ID=1: $1 \times 2 = 2$ (ambos avançam)

ID=2: $1 \times 1 = 1$ (ambos avançam)

Score final = 3

4.3 Requisito: Inserção dos Resultados na ABB

Após calcular o score de cada documento:

8. Chamar `ABB.inserir(id_documento, score)`
9. Documentos com score = 0 podem ser ignorados (não inserir na árvore)
10. A ABB organiza automaticamente os documentos por score

Exemplo:

Doc1 score=3, Doc2 score=1, Doc3 score=5

Inserção na ABB:

```
      (Doc1, 3)
     /      \
(Doc2, 1)   (Doc3, 5)
```

4.4 Requisito: Exibição dos Resultados

Percorrer a ABB em ordem decrescente de score (percurso in-ordem reverso: direita → raiz → esquerda) e exibir os documentos do mais ao menos relevante:

Resultado para a consulta "dados estrutura":

1. Doc3 (score = 5)
2. Doc1 (score = 3)
3. Doc2 (score = 1)

Para cada resultado exibir:

- Posição no ranking
- Nome do arquivo (nome_arquivo do Documento)
- Score de similaridade

5. Integração entre Estruturas

As estruturas não operam de forma isolada. A tabela abaixo resume como elas se comunicam:

De	Para	Dado Trocado
Fila ETL	Pilha de Transformações	Documento a processar (dequeue → push)
Pilha de Transformações	Etapa B	Lista de tokens (topo da pilha)
Tokens (ETL)	Tabela Hash	Palavra → busca/insere ID global
Tabela Hash	Lista Estática	ID gerado → armazena palavra no índice ID
Tabela Hash	Lista Encadeada do Doc	ID global → inserir/incrementar nó na lista esparsa
Lista Estática	Saída para usuário	ID → recupera texto legível da palavra
Listas Encadeadas (docs)	Cálculo de Score	Pares (id, freq) para produto escalar
Score calculado	ABB	Inserir (id_documento, score) na árvore
ABB	Saída para usuário	Percurso decrescente → ranking de resultados

6. Ordem Recomendada de Implementação

Siga rigorosamente esta ordem para evitar dependências não resolvidas:

Ordem	O que implementar	Depende de
1	Fila (Queue)	Nada
2	Pilha (Stack)	Nada
3	Lista Encadeada com Sentinela	Nada
4	Tabela Hash (usa lista encadeada para colisões)	Lista Encadeada (item 3)
5	Lista Estática (vocabulário global)	Nada
6	ABB por score	Nada
7	ETL completo (leitura, transformações, recursão)	Fila (1), Pilha (2)
8	Construção do vocabulário global	Tabela Hash (4), Lista Estática (5), ETL (7)

Ordem	O que implementar	Depende de
9	Representação esparsa dos documentos	Lista Encadeada (3), Vocabulário (8)
10	Motor de busca: processar consulta	ETL (7), Tabela Hash (4)
11	Cálculo de score (produto escalar)	Listas esparsas (9), Consulta (10)
12	Inserção e recuperação na ABB	ABB (6), Score (11)
13	Exibição dos resultados	ABB (12), Lista Estática (5)

7. Checklist de Requisitos Funcionais

Etapa A — ETL

- RF-A1: O sistema deve ler arquivos .txt e enfileirá-los na Fila
- RF-A2: O sistema deve processar documentos na ordem FIFO
- RF-A3: Cada documento deve passar por: minúsculas → sem pontuação → tokenização (com push na Pilha em cada etapa)
- RF-A4: A função de remoção de duplicados adjacentes deve ser implementada de forma recursiva

Etapa B — Representação

- RF-B1: Toda nova palavra deve ser inserida na Tabela Hash com um ID global único e sequencial
- RF-B2: A Lista Estática deve armazenar o mapeamento ID → palavra para todo o vocabulário
- RF-B3: Cada documento deve ser representado como uma Lista Encadeada com Sentinela de pares (id_termo, frequência)
- RF-B4: A lista esparsa de cada documento deve ser mantida em ordem crescente de id_termo
- RF-B5: Colisões na Tabela Hash devem ser tratadas por encadeamento (lista ligada no bucket)

Etapa C — Motor de Busca

- RF-C1: A consulta do usuário deve passar pelo mesmo pipeline ETL dos documentos
- RF-C2: Tokens da consulta não encontrados no vocabulário devem ser ignorados
- RF-C3: O score de similaridade deve ser calculado usando produto escalar entre vetores esparsos

- RF-C4: O algoritmo de produto escalar deve percorrer as duas listas simultaneamente, sem força bruta
- RF-C5: Os documentos devem ser inseridos na ABB usando o score como chave
- RF-C6: O resultado deve ser exibido em ordem decrescente de score
- RF-C7: Documentos com score = 0 não devem aparecer no resultado
- RF-C8: A lista encadeada temporária da consulta deve ser liberada ao final da busca

8. Referência Conceitual

Conceito	O que é	Onde aparece no projeto
TF (Term Frequency)	Número de vezes que um termo aparece em um documento	Frequência nos nós das listas esparsas
Vetor Esperso	Representação que armazena apenas os elementos não-nulos	Listas encadeadas dos documentos e da consulta
Produto Escalar	Soma dos produtos de termos em comum entre dois vetores	Cálculo do score de similaridade (Etapa C)
Busca Vetorial	Comparação de textos via operações matemáticas entre vetores	Todo o fluxo da Etapa C
Índice Invertido	Estrutura que mapeia termos para os documentos que os contêm	Tabela Hash + Listas Esparsas (Etapa B)

Nota sobre extensão: O projeto utiliza TF simples (frequência bruta). Uma extensão natural é o TF-IDF, que penaliza termos muito comuns em todos os documentos, aumentando a precisão da busca.